Experiment 28: Deep Boltzmann Machine / Convolutional Boltzmann Machine Simulation

Question

Problem Statement: A textile company wants to model and generate fabric texture patterns. Spatial structure matters, so a fully connected RBM is not enough. Small grayscale texture patches or MNIST images. The objective is to simulate a two-layer DBM or a convolutional RBM and study its representations. Use NumPy, SciPy, TensorFlow, Keras, Matplotlib.

Tasks:

1. Build a two-hidden-layer DBM with mean-field inference, or a convolutional RBM with shared filters.
2. Train with approximate learning (mean-field plus Gibbs sampling).
3. Visualise filters and hidden-unit activations.
4. Generate samples and reconstructions.
5. Compare with the single-layer RBM from Experiment 25 on reconstruction quality.

Code

```
"""Experiment 28: Deep Boltzmann Machine Simulation (two hidden layers, mean-field + Gibbs)"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import expit

rng = np.random.RandomState(0)

def load_mnist(n_train=8000, n_test=1000):
    try:
        from tensorflow import keras
        (x, _), (xt, _) = keras.datasets.mnist.load_data()
        x = x.reshape(-1, 784) / 255.0; xt = xt.reshape(-1, 784) / 255.0
    except Exception as e:
        print("WARNING: MNIST not available (", type(e).__name__, ") -> using scikit-learn digits scaled to
28x28.")
        from sklearn.datasets import load_digits
        from scipy.ndimage import zoom
        d = load_digits().images / 16.0
        d = np.stack([zoom(im, 3.5, order=1) for im in d]).reshape(-1, 784)
        x, xt = d[:int(0.8 * len(d))], d[int(0.8 * len(d)):]
    return (x[:n_train] > 0.5).astype(float), (xt[:n_test] > 0.5).astype(float)

X_train, X_test = load_mnist()
H1, H2, BATCH = 200, 100, 100
print("Train:", X_train.shape, "Test:", X_test.shape)

def sample(p): return (rng.rand(*p.shape) < p).astype(float)

class RBM:                                                    # used for greedy initialisation and as
the baseline
    def __init__(self, nv, nh):
        self.W = rng.normal(0, 0.01, (nv, nh)); self.a = np.zeros(nv); self.b = np.zeros(nh)
    def p_h(self, v): return expit(v @ self.W + self.b)
    def p_v(self, h): return expit(h @ self.W.T + self.a)
    def fit(self, X, epochs, lr=0.05):
        for _ in range(epochs):
            perm = rng.permutation(len(X))
            for i in range(0, len(X), BATCH):
                v0 = X[perm[i:i + BATCH]]; ph0 = self.p_h(v0); pv1 = self.p_v(sample(ph0)); ph1 =
self.p_h(pv1)
                self.W += lr * (v0.T @ ph0 - pv1.T @ ph1) / len(v0)
                self.a += lr * np.mean(v0 - pv1, axis=0); self.b += lr * np.mean(ph0 - ph1, axis=0)

PRE_EPOCHS, DBM_EPOCHS = 5, 10

# ---------------- Task 1a: greedy pre-training of the two layers ----------------
rbm1 = RBM(784, H1); rbm1.fit(X_train, PRE_EPOCHS)
```

```
rbm2 = RBM(H1, H2); rbm2.fit(sample(rbm1.p_h(X_train)), PRE_EPOCHS)


# ---------------- Task 1b: DBM with mean-field inference ----------------
# Energy: E(v,h1,h2) = -v'W1 h1 - h1'W2 h2 - a'v - b1'h1 - b2'h2
class DBM:
    def __init__(self, rbm1, rbm2):
        self.W1, self.a, self.b1 = rbm1.W.copy(), rbm1.a.copy(), rbm1.b.copy()
        self.W2, self.b2 = rbm2.W.copy(), rbm2.b.copy()
        self.particles = None

    def mean_field(self, v, iters=10):                              # approximate posterior q(h1,h2|v) = prod
of Bernoulli(mu)
        mu1 = expit(v @ self.W1 + self.b1); mu2 = expit(mu1 @ self.W2 + self.b2)
        for _ in range(iters):
            mu1 = expit(v @ self.W1 + mu2 @ self.W2.T + self.b1)    # bottom-up AND top-down input
            mu2 = expit(mu1 @ self.W2 + self.b2)
        return mu1, mu2

    def gibbs_step(self, v, h1, h2):                                # alternate layers: h1 | v,h2  then  v,h2
| h1
        h1 = sample(expit(v @ self.W1 + h2 @ self.W2.T + self.b1))
        v = sample(expit(h1 @ self.W1.T + self.a))
        h2 = sample(expit(h1 @ self.W2 + self.b2))
        return v, h1, h2

    # ---- Task 2: approximate learning = mean-field (positive phase) + persistent Gibbs chains (negative
phase) ----
    def train_batch(self, v, lr):
        mu1, mu2 = self.mean_field(v)
        if self.particles is None:
            self.particles = [sample(np.full((len(v), 784), 0.5)), sample(mu1), sample(mu2)]
        vp, h1p, h2p = self.gibbs_step(*self.particles); self.particles = [vp, h1p, h2p]
        B = len(v)
        self.W1 += lr * (v.T @ mu1 - vp.T @ h1p) / B
        self.W2 += lr * (mu1.T @ mu2 - h1p.T @ h2p) / B
        self.a += lr * np.mean(v - vp, axis=0)
        self.b1 += lr * np.mean(mu1 - h1p, axis=0); self.b2 += lr * np.mean(mu2 - h2p, axis=0)

    def reconstruct(self, v):
        mu1, _ = self.mean_field(v)
        return expit(mu1 @ self.W1.T + self.a)


dbm = DBM(rbm1, rbm2)
rbm_base = RBM(784, H1); rbm_base.fit(X_train, PRE_EPOCHS + DBM_EPOCHS)     # single-layer baseline, same
total epochs
mse = lambda V, R: np.mean((V - R) ** 2)
hist = [mse(X_test, dbm.reconstruct(X_test))]
for ep in range(1, DBM_EPOCHS + 1):
    perm = rng.permutation(len(X_train))
    for i in range(0, len(X_train) - BATCH + 1, BATCH):              # full batches (the particle set has a
fixed size)
        dbm.train_batch(X_train[perm[i:i + BATCH]], lr=0.005)
    hist.append(mse(X_test, dbm.reconstruct(X_test)))
    print(f"DBM epoch {ep:2d}: test reconstruction MSE = {hist[-1]:.4f}")


plt.figure(figsize=(6.5, 4)); plt.plot(range(0, DBM_EPOCHS + 1), hist, "o-"); plt.xlabel("DBM training
epoch")
plt.ylabel("Test reconstruction MSE"); plt.title("DBM reconstruction error"); plt.grid(alpha=0.3); plt.show()


# ---------------- Task 3: filters and hidden-unit activations ----------------
fig, axes = plt.subplots(6, 10, figsize=(12, 7.5))
for i, a in enumerate(axes.ravel()):
    a.imshow(dbm.W1[:, i].reshape(28, 28), cmap="gray"); a.axis("off")
plt.suptitle("DBM first-layer filters (W1)"); plt.tight_layout(); plt.show()

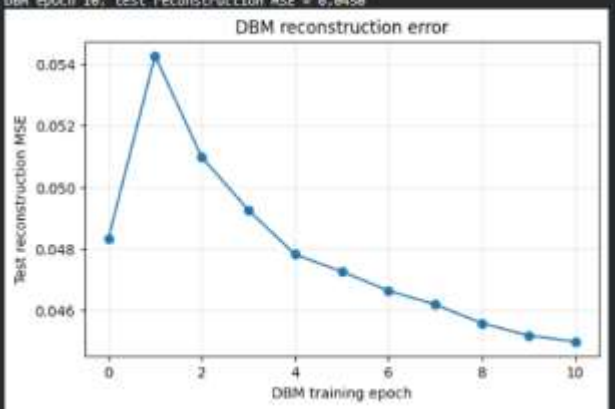

mu1, mu2 = dbm.mean_field(X_test[:10])
fig, ax = plt.subplots(1, 2, figsize=(13, 3.5))
ax[0].imshow(mu1, aspect="auto", cmap="viridis"); ax[0].set_title("Layer-1 hidden activations (10 test digits
x 200 units)")
ax[1].imshow(mu2, aspect="auto", cmap="viridis"); ax[1].set_title("Layer-2 hidden activations (10 test digits
x 100 units)")
for a in ax: a.set_ylabel("Digit")
plt.tight_layout(); plt.show()
```

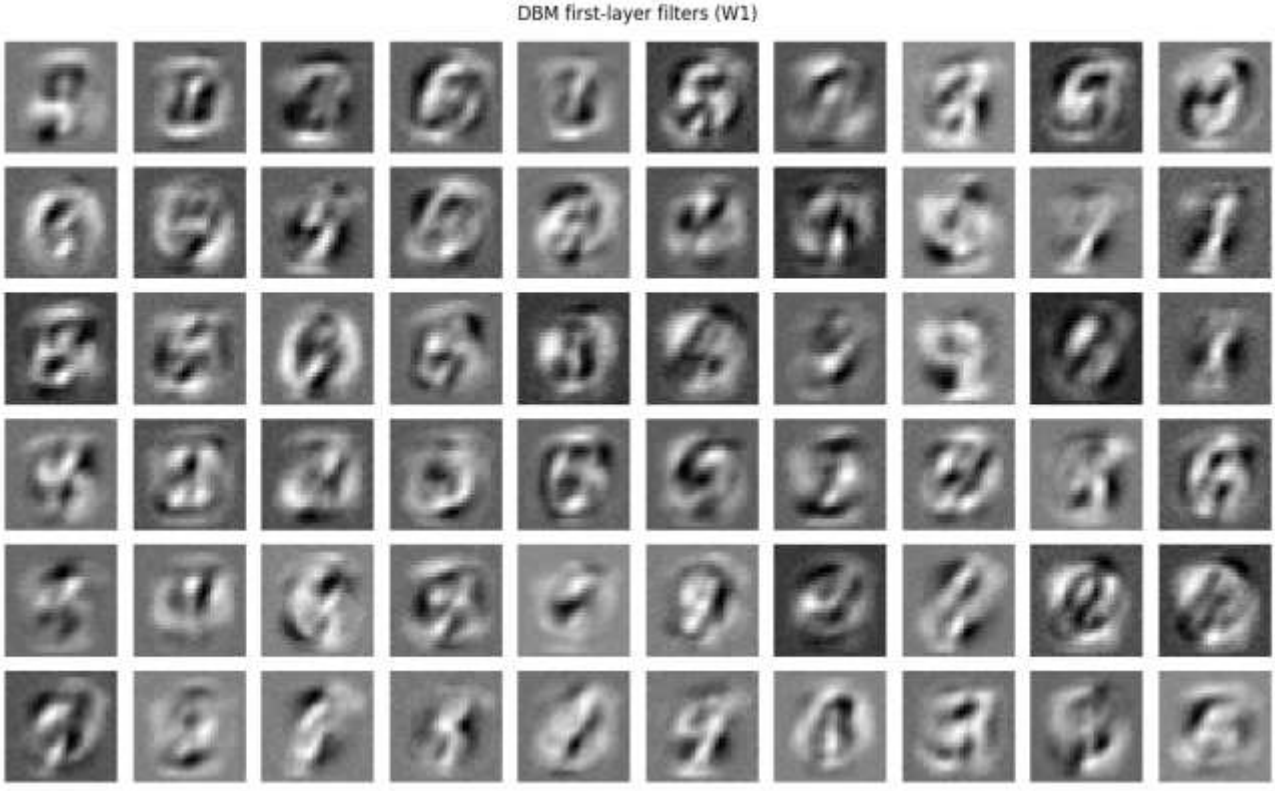
```
# ---------------- Task 4: samples and reconstructions ----------------
v, h1, h2 = dbm.particles
for _ in range(500):
    v, h1, h2 = dbm.gibbs_step(v, h1, h2)
fig, axes = plt.subplots(3, 10, figsize=(14, 4.6))
rec = dbm.reconstruct(X_test[:10])
for i in range(10):
    axes[0, i].imshow(X_test[i].reshape(28, 28), cmap="gray"); axes[1, i].imshow(rec[i].reshape(28, 28),
cmap="gray")
    axes[2, i].imshow(expit(h1[i] @ dbm.W1.T + dbm.a).reshape(28, 28), cmap="gray")
    for r in range(3): axes[r, i].axis("off")
axes[0, 0].set_title("Original", loc="left"); axes[1, 0].set_title("DBM reconstruction", loc="left"); axes[2,
0].set_title("Samples generated from the DBM", loc="left")
plt.tight_layout(); plt.show()

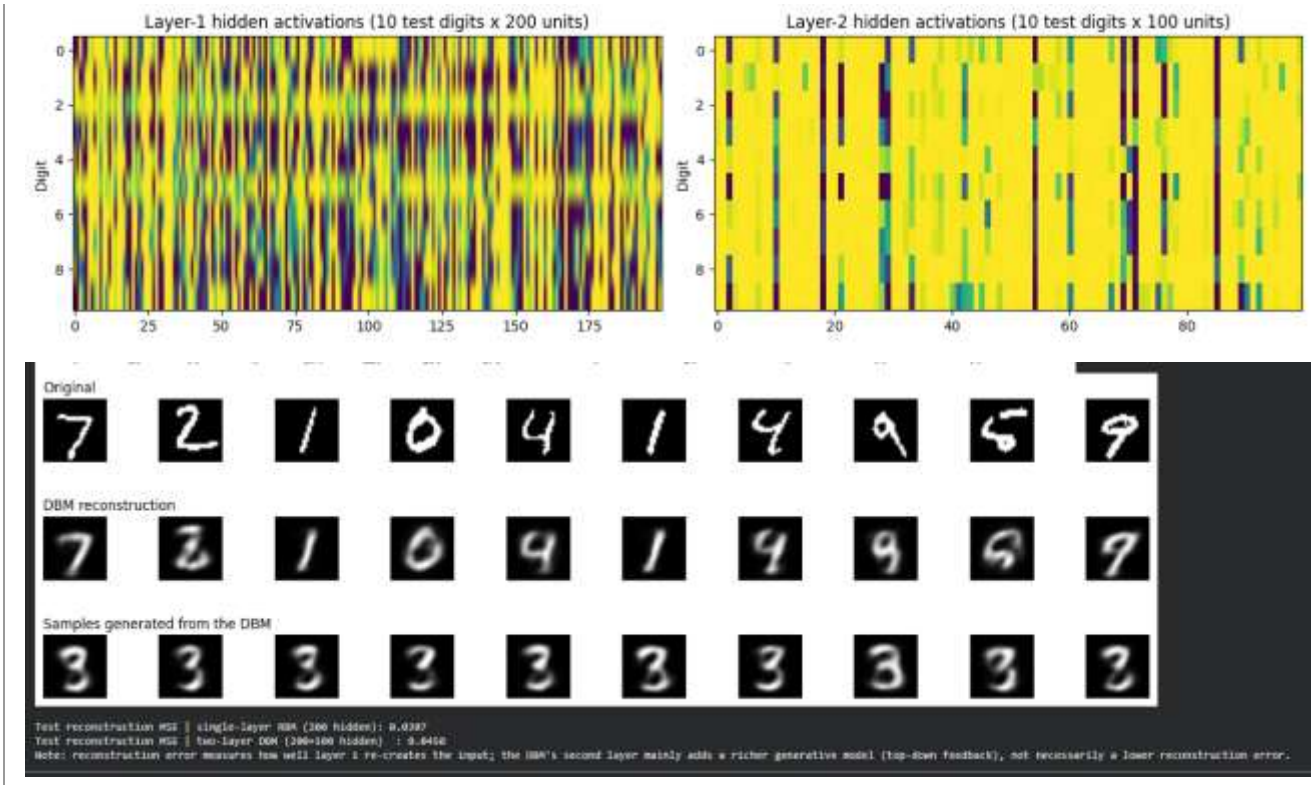

# ---------------- Task 5: compare with a single-layer RBM ----------------
rec_dbm = mse(X_test, dbm.reconstruct(X_test)); rec_rbm = mse(X_test, rbm_base.p_v(rbm_base.p_h(X_test)))
print(f"\nTest reconstruction MSE | single-layer RBM ({H1} hidden): {rec_rbm:.4f}")
print(f"Test reconstruction MSE | two-layer DBM ({H1}+{H2} hidden)  : {rec_dbm:.4f}")
print("Note: reconstruction error measures how well layer 1 re-creates the input; the DBM's second layer "
      "mainly adds a richer generative model (top-down feedback), not necessarily a lower reconstruction
error.")
```

Output

```
Train: (8000, 784) Test: (1000, 784)
DBM epoch  1: test reconstruction MSE = 0.0543
DBM epoch  2: test reconstruction MSE = 0.0510
DBM epoch  3: test reconstruction MSE = 0.0493
DBM epoch  4: test reconstruction MSE = 0.0479
DBM epoch  5: test reconstruction MSE = 0.0473
DBM epoch  6: test reconstruction MSE = 0.0466
DBM epoch  7: test reconstruction MSE = 0.0462
DBM epoch  8: test reconstruction MSE = 0.0456
DBM epoch  9: test reconstruction MSE = 0.0452
DBM epoch 10: test reconstruction MSE = 0.0450
```






Layer-1 hidden activations (10 test digits x 200 units)
Digit
0
2
4
6
8
0
25
50
75
100
125
150
175
Layer-2 hidden activations (10 test digits x 100 units)
Digit
0
2
4
6
8
0
20
40
60
80
Original
DBM reconstruction
Samples generated from the DBM
Test reconstruction MSE | single-layer RBM (200 hidden): 0.0387
Test reconstruction MSE | two-layer DBM (200→100 hidden) : 0.0450
Note: reconstruction error measures how well layer 1 re-creates the input; the DBM's second layer mainly adds a richer generative model (top-down feedback), not necessarily a lower reconstruction error.